

Лабораторная работа № 7.

Связанные данные и запросы в Prisma

Тема: Получение связанных данных через include, select и relation filters

Цель: Научиться получать связанные данные Product, Category, Order и OrderItem через Prisma Client, формировать вложенные JSON-ответы и фильтровать записи по связанным моделям.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом express-shop. В проекте уже должны использоваться PostgreSQL, Prisma Client и архитектура Route → Controller → Service.

В базе данных должны существовать связанные таблицы categories, products, users, orders и order_items.

Связи, используемые в работе

Связь	Кардинальность	Пример
Category → Product	1:N	Одна категория содержит много товаров
User → Order	1:N	Один пользователь может иметь много заказов
Order → OrderItem	1:N	Один заказ содержит несколько позиций
Product → OrderItem	1:N	Один товар может встречаться во многих заказах

1. Проверьте модели Prisma

Откройте prisma/schema.prisma и убедитесь, что между моделями определены relation-поля.

Фрагмент schema.prisma

```
model Category {
  id Int @id @default(autoincrement())
  name String @unique @db.VarChar(100)
  products Product[]

  @@map("categories")
}

model Product {
  id Int @id @default(autoincrement())
  name String @db.VarChar(150)
  price Int
  categoryId Int @map("category_id")
  inStock Boolean @default(true) @map("in_stock")
  category Category @relation(fields: [categoryId], references: [id])
  orderItems OrderItem[]
}
```

```
    @@map("products")
  }
}
```

Связи Order и OrderItem

```
model User {
  id Int @id @default(autoincrement())
  name String
  email String @unique
  orders Order[]

  @@map("users")
}

model Order {
  id Int @id @default(autoincrement())
  userId Int @map("user_id")
  status String @db.VarChar(30)
  createdAt DateTime @default(now()) @map("created_at")
  user User @relation(fields: [userId], references: [id])
  items OrderItem[]

  @@map("orders")
}

model OrderItem {
  orderId Int @map("order_id")
  productId Int @map("product_id")
  quantity Int
  price Int
  order Order @relation(fields: [orderId], references: [id])
  product Product @relation(fields: [productId], references: [id])

  @@id([orderId, productId])
  @@map("order_items")
}
```

Важно

Названия полей должны соответствовать вашей фактической схеме. Если prisma db pull создал другие имена, используйте имена из вашего проекта.

2. Сгенерируйте Prisma Client

PowerShell

```
npx prisma@7 generate
```

После изменения schema.prisma Prisma Client необходимо сгенерировать повторно.

3. Получите Product вместе с Category

Обычный findMany() возвращает скалярные поля Product, но не добавляет связанные модели автоматически.

services/productService.js

```
export async function getAllProducts() {
  return prisma.product.findMany({
    include: {
      category: true
    },
    orderBy: {
      id: "asc"
    }
  });
}
```

Теперь каждый объект Product содержит вложенный объект category.

Пример ответа

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 420000,
    "categoryId": 1,
    "inStock": true,
    "category": {
      "id": 1,
      "name": "Ноутбуки"
    }
  }
]
```

4. Ограничьте поля через вложенный select

Если клиенту не нужен весь объект Category, запросите только необходимые поля.

services/productService.js

```
export async function getAllProducts() {
  return prisma.product.findMany({
    select: {
      id: true,
      name: true,
      price: true,
      inStock: true,
      category: {
        select: {
          id: true,
          name: true
        }
      }
    },
    orderBy: {
      id: "asc"
    }
  });
}
```

Практический принцип

Возвращайте из API только те поля и связи, которые действительно нужны frontend.

5. Получите один Product со связанной Category

```
services/productService.js

export async function findProductById(id) {
  return prisma.product.findUnique({
    where: {
      id
    },
    include: {
      category: true
    }
  });
}
```

Проверка Product API

Метод	URL	Ожидаемый результат
GET	/api/products/1	Product с вложенным объектом category
GET	/api/products/999999	404 Product not found

6. Создайте Order Service

Создайте файл services/orderService.js. Сервис будет получать заказ вместе с пользователем, позициями заказа и товарами.

```
services/orderService.js

import { prisma } from "../config/prisma.js";

export async function findOrderById(id) {
  return prisma.order.findUnique({
    where: {
      id
    },
    include: {
      user: {
        select: {
          id: true,
          name: true,
          email: true
        }
      },
      items: {
        include: {
          product: {
            select: {
              id: true,
              name: true
            }
          }
        }
      }
    }
  });
}
```

```
}  
}  
}  
});  
}
```

7. Изучите структуру результата

Пример вложенного результата

```
{  
  "id": 10,  
  "userId": 3,  
  "status": "new",  
  "createdAt": "2026-09-21T10:00:00.000Z",  
  "user": {  
    "id": 3,  
    "name": "Alex",  
    "email": "alex@example.com"  
  },  
  "items": [  
    {  
      "orderId": 10,  
      "productId": 1,  
      "quantity": 2,  
      "price": 420000,  
      "product": {  
        "id": 1,  
        "name": "Laptop"  
      }  
    }  
  ]  
}
```

Prisma преобразует связанные строки PostgreSQL в обычный вложенный JavaScript-объект.

8. Рассчитайте сумму заказа

Цена позиции хранится в `OrderItem.price`. Для суммы заказа умножьте цену позиции на количество и сложите результаты.

```
services/orderService.js  
  
export async function getOrderDetails(id) {  
  const order = await findOrderById(id);  
  
  if (!order) {  
    return null;  
  }  
  
  const total = order.items.reduce(  
    (sum, item) => sum + item.price * item.quantity,  
    0  
  );  
  
  return {  
    ...order,  
    total  
  };  
}
```

```
}
```

Цена заказа

Для расчёта истории заказа используйте `OrderItem.price`, а не текущий `Product.price`. Цена товара в каталоге может измениться после оформления заказа.

9. Создайте Order Controller

```
controllers/orderController.js

import { getOrderDetails } from "../services/orderService.js";
import { AppError } from "../utils/AppError.js";

export async function getOrderById(req, res, next) {
  try {
    const id = Number(req.params.id);
    const order = await getOrderDetails(id);

    if (!order) {
      return next(new AppError("Order not found", 404));
    }

    res.json(order);
  } catch (error) {
    next(error);
  }
}
```

10. Создайте маршрут заказа

```
routes/orderRoutes.js

import { Router } from "express";
import { getOrderById } from "../controllers/orderController.js";

const router = Router();

router.get("/:id", getOrderById);

export default router;
```

Подключите маршрутизатор заказов к приложению.

```
app.js

import orderRoutes from "../routes/orderRoutes.js";

app.use("/api/orders", orderRoutes);
```

Order API

Метод	URL	Назначение
GET	/api/orders/10	Получить заказ со связанным User, OrderItem и Product
GET	/api/orders/999999	Проверить ответ 404

11. Отфильтруйте заказы по связанным позициям

Добавьте функцию, которая возвращает заказы, содержащие указанный товар.

```
services/orderService.js

export async function findOrdersByProduct(productId) {
  return prisma.order.findMany({
    where: {
      items: {
        some: {
          productId
        }
      }
    },
    include: {
      items: {
        include: {
          product: true
        }
      }
    },
    orderBy: {
      id: "asc"
    }
  });
}
```

Оператор `some` означает: вернуть `Order`, если хотя бы одна связанная запись `OrderItem` соответствует условию.

12. Проверьте relation filter

Для проверки используйте существующий `productId` из вашей базы данных.

```
Пример вызова

const orders = await findOrdersByProduct(1);

console.log(orders);
```

Relation filter

Для связей «один ко многим» Prisma также предоставляет `every` и `none`. В этой лабораторной основной практический оператор — `some`.

13. Самостоятельное задание

1. Измените `getAllProducts()`, чтобы товары возвращались вместе с названием категории.

2. Для списка товаров используйте вложенный select, чтобы Category возвращала только id и name.
3. Реализуйте GET /api/orders/:id с User, OrderItem и Product.
4. Добавьте вычисляемое поле total в ответ заказа.
5. Реализуйте функцию поиска заказов, содержащих определённый Product, через relation filter some.
6. Проверьте корректную обработку отсутствующего Product и Order.
7. Проверьте результат через HTTP-клиент и Prisma Studio.

14. Проверка Prisma Studio

```
PowerShell  
  
npx prisma@7 studio
```

Сравните связи и значения в Prisma Studio с JSON, который возвращает REST API.

Требования к результату

- Product API возвращает связанную Category.
- Для ограничения вложенных полей используется select.
- Order API возвращает связанного User.
- Order API возвращает массив связанных OrderItem.
- Каждый OrderItem содержит данные связанного Product.
- В ответ заказа добавляется вычисляемое поле total.
- Поиск заказов по товару использует relation filter some.
- Prisma-запросы находятся в Service, а не в Controller или Route.
- Отсутствующие записи корректно приводят к HTTP 404.
- Данные фактически читаются из PostgreSQL.

Критерии оценивания

№	Критерий
1	Связи Prisma Schema определены корректно
2	Prisma Client успешно генерируется
3	Product возвращается вместе с Category
4	Используется вложенный select
5	Order возвращается вместе с User
6	Order содержит связанные OrderItem
7	OrderItem содержит связанный Product
8	Поле total рассчитывается корректно
9	Relation filter some работает корректно
10	Несуществующий Product обрабатывается как 404

№	Критерий
11	Несуществующий Order обрабатывается как 404
12	Prisma-запросы размещены в Service
13	REST endpoints работают через Controller
14	Результаты проверены через HTTP-запросы
15	Студент может объяснить различие include и select